

Spring Boot 应用集成企业LDAP登录详细设计方案

1. 现有体系与LDAP集成关系梳理

- 现有体系：应用内有用户表（User）、角色表（Role）、权限表（Permission），通过本地账号密码认证，用户与角色、权限有多对多关系。
- LDAP集成目标：允许用户既可用本地账号登录，也可用企业LDAP账号登录。LDAP用户首次登录时自动同步/注册到本地用户表，并可基于LDAP组自动分配本地角色。

2. 设计目标与集成原则

- 兼容双重认证：支持本地账号和LDAP账号共存，登录时自动区分。
- 用户信息同步：LDAP用户首次登录时自动注册本地账号，后续登录同步关键信息（如邮箱、姓名、部门等）。
- 权限映射：可配置LDAP组与本地角色的映射关系，实现权限自动分配。
- 安全合规：所有LDAP通信必须加密，服务账号最小权限，防止LDAP注入。
- 可扩展性：后续可支持更多认证方式（如OAuth、CAS等）。

2.1 多企业可选启用LDAP与动态配置

- 应用需支持“每个企业可独立选择是否启用LDAP登录”，并能为每个企业单独配置LDAP参数。
- 推荐做法：
 - 在企业租户表（如Tenant）中增加LDAP相关配置字段（如是否启用、LDAP服务器地址、Base DN、Bind DN、密码等）。
 - 应用启动时加载所有企业的LDAP配置，或按需动态加载。
 - 登录时根据用户所属企业动态判断是否走LDAP认证及用哪个配置。

单企业数据库存储方式

- 适用于每套部署只服务一个企业，LDAP配置存储在数据库表（如 `ldap_config`），支持管理后台动态维护。
- 表结构示例：

```
CREATE TABLE ldap_config (  
  id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  enabled BOOLEAN NOT NULL DEFAULT FALSE,  
  url VARCHAR(255) NOT NULL,  
  base_dn VARCHAR(255) NOT NULL,  
  bind_dn VARCHAR(255) NOT NULL,  
  password VARCHAR(255) NOT NULL,  
  user_pattern VARCHAR(255),  
  group_base VARCHAR(255),  
  group_filter VARCHAR(255),  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP  
);
```

- 仅有一条有效配置，或用 `id=1` 约定。

3. 技术方案与认证流程

3.1 认证流程

1. 用户在登录页输入用户名和密码。
2. 系统判断用户名是否为本地账号（如有本地标记字段），优先本地认证。
3. 若本地认证失败或为LDAP账号，则尝试LDAP认证：
 - 用服务账号（Bind DN）连接LDAP，查找用户DN。
 - 用用户DN和明文密码尝试Bind。
 - 成功则认证通过。
4. 若LDAP认证通过：
 - 查询LDAP属性（如mail、displayName、memberOf等）。
 - 若本地无此用户，则自动注册。
 - 同步用户信息，分配本地角色。
5. 认证通过后，生成Spring Security认证会话。

3.2 用户同步与权限映射

- 用户同步：LDAP用户首次登录时，自动在本地User表插入记录，后续登录同步关键信息。
- 权限映射：通过配置文件或数据库，维护LDAP组与本地角色的映射表。登录时根据LDAP的memberOf属性自动分配本地角色。

4. Spring Security 配置与实现代码

4.1 依赖引入

```

<!-- pom.xml -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.ldap</groupId>
    <artifactId>spring-ldap-core</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.security</groupId>
    <artifactId>spring-security-ldap</artifactId>
</dependency>

```

4.2 application.yml 配置示例

```

spring:
  ldap:
    urls: ldap://ldap.example.com:389
    base: dc=example,dc=com
    username: cn=ldap-reader,ou=ServiceAccounts,dc=example,dc=com # 服务账号
    password: your_ldap_password
    # 若用LDAPS, urls: ldaps://ldap.example.com:636
  security:
    user-dn-patterns: "uid={0},ou=Users"
    group-search-base: "ou=Groups"
    group-search-filter: "(member={0})"

ldap:
  group-role-mapping:
    "cn=研发部,ou=Groups,dc=example,dc=com": "DEV"
    "cn=管理员,ou=Groups,dc=example,dc=com": "ADMIN"

```

4.3 Spring Security 配置

```

@Configuration
@EnableWebSecurity
public class SecurityConfig {
    @Autowired
    private LdapUserDetailsService ldapUserDetailsService;
    @Autowired
    private LocalUserDetailsService localUserDetailsService;

    @Bean
    public AuthenticationManager authenticationManager(HttpSecurity http) throws Exception
    {
        return http.getSharedObject(AuthenticationManagerBuilder.class)
            .userDetailsService(localUserDetailsService) // 本地优先
            .and()

```

```

        .authenticationProvider(ldapAuthProvider())
        .build();
    }

    @Bean
    public AuthenticationProvider ldapAuthProvider() {
        LdapAuthenticationProvider provider = new LdapAuthenticationProvider(
            ldapAuthenticator(), ldapAuthoritiesPopulator()
        );
        provider.setUserDetailsContextMapper(ldapUserDetailsMapper());
        return provider;
    }

    @Bean
    public BindAuthenticator ldapAuthenticator() {
        DefaultSpringSecurityContextSource contextSource = new
DefaultSpringSecurityContextSource(
            "ldap://ldap.example.com:389/dc=example,dc=com");
        contextSource.setUserDn("cn=ldap-reader,ou=ServiceAccounts,dc=example,dc=com");
        contextSource.setPassword("your_ldap_password");
        contextSource.afterPropertiesSet();
        BindAuthenticator authenticator = new BindAuthenticator(contextSource);
        authenticator.setUserDnPatterns(new String[] {"uid={0},ou=Users"});
        return authenticator;
    }

    @Bean
    public LdapAuthoritiesPopulator ldapAuthoritiesPopulator() {
        return new CustomLdapAuthoritiesPopulator();
    }

    @Bean
    public UserDetailsContextMapper ldapUserDetailsMapper() {
        return new CustomLdapUserDetailsMapper();
    }
}

```

4.4 用户同步与角色映射实现

```

// 用户信息同步与角色分配
public class CustomLdapUserDetailsMapper implements UserDetailsContextMapper {
    @Autowired
    private UserRepository userRepository;
    @Autowired
    private RoleRepository roleRepository;
    @Value("${ldap.group-role-mapping}")
    private Map<String, String> groupRoleMapping;

    @Override
    public UserDetails mapUserFromContext(DirContextOperations ctx, String username,
Collection<? extends GrantedAuthority> authorities) {

```

```

// 1. 读取LDAP属性
String email = ctx.getStringAttribute("mail");
String displayName = ctx.getStringAttribute("displayName");
String[] groups = ctx.getStringAttributes("memberOf");

// 2. 本地用户同步
User user = userRepository.findByUsername(username)
    .orElseGet(() -> new User(username, email, displayName));
user.setEmail(email);
user.setDisplayName(displayName);
userRepository.save(user);

// 3. 角色映射
Set<Role> roles = new HashSet<>();
if (groups != null) {
    for (String groupDn : groups) {
        String roleName = groupRoleMapping.get(groupDn);
        if (roleName != null) {
            Role role = roleRepository.findByName(roleName)
                .orElseGet(() -> roleRepository.save(new Role(roleName)));
            roles.add(role);
        }
    }
}
user.setRoles(roles);
userRepository.save(user);

// 4. 返回Spring Security UserDetails
return new org.springframework.security.core.userdetails.User(
    user.getUsername(), "N/A", roles.stream().map(r -> new
SimpleGrantedAuthority(r.getName())).toList()
);
}

@Override
public void mapUserToContext(UserDetails user, DirContextAdapter ctx) {
    // 不需要实现
}
}

```

4.5 兼容本地账号登录

- 本地账号登录通过 `LocalUserDetailsService` 实现，优先本地认证，失败后再尝试LDAP。
- 可在User表加字段 `authType` 区分本地/LDAP用户。

4.6 LDAP用户信息与注销定时同步

为保证本地用户信息与LDAP目录实时一致，建议实现定时批量同步机制：

- 定时任务：**定期（如每天凌晨）批量拉取LDAP所有用户信息，遍历同步到本地数据库。

- **注销同步**：对比本地与LDAP用户列表，若本地存在但LDAP已无该用户，则自动将本地用户设为禁用（如 `enabled=false`），实现离职/注销同步。
- **属性更新**：同步时更新本地用户的邮箱、姓名、部门等关键信息。
- **安全建议**：定时任务使用只读服务账号，且同步范围仅限业务相关OU。

Spring定时任务代码示例：

```
@Component
public class LdapUserSyncTask {
    @Autowired
    private LdapTemplate ldapTemplate;
    @Autowired
    private UserRepository userRepository;

    @Scheduled(cron = "0 0 2 * * ?") // 每天凌晨2点
    public void syncAllLdapUsers() {
        List<LdapUser> ldapUsers = ldapTemplate.search(
            "ou=Users,dc=example,dc=com",
            "(objectClass=person)",
            new LdapUserAttributesMapper()
        );
        Set<String> ldapUsernames =
            ldapUsers.stream().map(LdapUser::getUsername).collect(Collectors.toSet());
        // 1. 同步/更新LDAP用户信息
        for (LdapUser ldapUser : ldapUsers) {
            User user = userRepository.findByUsername(ldapUser.getUsername())
                .orElseGet(() -> new User(ldapUser.getUsername(), ...));
            user.setEmail(ldapUser.getEmail());
            user.setDisplayName(ldapUser.getDisplayName());
            // ... 其他属性
            user.setEnabled(true);
            userRepository.save(user);
        }
        // 2. 注销本地已不存在的LDAP用户
        List<User> localLdapUsers = userRepository.findAllLdapUsers();
        for (User user : localLdapUsers) {
            if (!ldapUsernames.contains(user.getUsername())) {
                user.setEnabled(false); // 或 user.setStatus("DISABLED")
                userRepository.save(user);
            }
        }
    }
}
```

4.7 单企业数据库配置与管理后台实现

1. 配置加载与使用

- 启动时或每次认证前，从 `ldap_config` 表加载配置。
- 可用Spring的 `@ConfigurationProperties` 或自定义Service加载。

代码示例：

```
@Entity
public class LdapConfig {
    @Id
    private Long id;
    private boolean enabled;
    private String url;
    private String baseDn;
    private String bindDn;
    private String password;
    private String userPattern;
    private String groupBase;
    private String groupFilter;
    // getter/setter
}

@Repository
public interface LdapConfigRepository extends JpaRepository<LdapConfig, Long> {
    Optional<LdapConfig> findTopByOrderByIdAsc();
}

@Service
public class LdapConfigService {
    @Autowired
    private LdapConfigRepository repo;
    public LdapConfig getConfig() {
        return repo.findTopByOrderByIdAsc().orElseThrow(...);
    }
}
```

- 在认证Provider中动态加载配置：

```
@Autowired
private LdapConfigService ldapConfigService;

LdapConfig config = ldapConfigService.getConfig();
if (config.isEnabled()) {
    // 构造LdapContextSource并认证
}
```

2. 管理后台设计要点

- 提供LDAP配置的增/查/改（一般只允许有一条配置）。
- 支持密码加密存储与解密显示（如用Jasypt或自定义加密工具）。
- 配置变更后可立即生效（可加缓存和刷新机制）。
- 管理后台界面字段：是否启用、LDAP地址、Base DN、Bind DN、密码、用户模式、组Base、组过滤等。

接口示例：

```

@RestController
@RequestMapping("/api/ldap-config")
public class LdapConfigController {
    @Autowired
    private LdapConfigRepository repo;

    @GetMapping
    public LdapConfig get() {
        return repo.findTopByOrderByIdAsc().orElse(null);
    }

    @PostMapping
    public LdapConfig save(@RequestBody LdapConfig config) {
        // 可加加密逻辑
        return repo.save(config);
    }
}

```

- 前端可用表单页面实现配置的增改查，保存后后端自动生效。

3. 管理后台前端页面示例

以下为基于Vue3 + Element Plus的LDAP配置管理表单页面代码示例，可直接对接前述后端接口：

```

<template>
  <el-form :model="form" :rules="rules" ref="formRef" label-width="120px" style="max-
width: 500px;">
    <el-form-item label="是否启用LDAP" prop="enabled">
      <el-switch v-model="form.enabled" />
    </el-form-item>
    <el-form-item label="LDAP地址" prop="url">
      <el-input v-model="form.url" placeholder="如 ldap://ldap.example.com:389" />
    </el-form-item>
    <el-form-item label="Base DN" prop="baseDn">
      <el-input v-model="form.baseDn" placeholder="如 dc=example,dc=com" />
    </el-form-item>
    <el-form-item label="Bind DN" prop="bindDn">
      <el-input v-model="form.bindDn" placeholder="如 cn=ldap-
reader,ou=ServiceAccounts,dc=example,dc=com" />
    </el-form-item>
    <el-form-item label="密码" prop="password">
      <el-input v-model="form.password" type="password" show-password />
    </el-form-item>
    <el-form-item label="用户模式" prop="userPattern">
      <el-input v-model="form.userPattern" placeholder="如 uid={0},ou=Users" />
    </el-form-item>
    <el-form-item label="组Base" prop="groupBase">
      <el-input v-model="form.groupBase" placeholder="如 ou=Groups" />
    </el-form-item>
    <el-form-item label="组过滤器" prop="groupFilter">
      <el-input v-model="form.groupFilter" placeholder="如 (member={0})" />
    </el-form-item>
  </el-form>
</template>

```



```

    </el-form-item>
    <el-form-item>
      <el-button type="primary" @click="onSave">保存</el-button>
      <el-button @click="onLoad">重载</el-button>
    </el-form-item>
  </el-form>
</template>

<script setup>
import { ref, reactive } from 'vue'
import { ElMessage } from 'element-plus'
import axios from 'axios'

const formRef = ref()
const form = reactive({
  enabled: false,
  url: '',
  baseDn: '',
  bindDn: '',
  password: '',
  userPattern: '',
  groupBase: '',
  groupFilter: ''
})

const rules = {
  url: [{ required: true, message: '请输入LDAP地址', trigger: 'blur' }],
  baseDn: [{ required: true, message: '请输入Base DN', trigger: 'blur' }],
  bindDn: [{ required: true, message: '请输入Bind DN', trigger: 'blur' }],
  password: [{ required: true, message: '请输入密码', trigger: 'blur' }]
}

function onLoad() {
  axios.get('/api/ldap-config').then(res => {
    Object.assign(form, res.data)
    ElMessage.success('加载成功')
  })
}

function onSave() {
  formRef.value.validate(valid => {
    if (!valid) return
    axios.post('/api/ldap-config', form).then(() => {
      ElMessage.success('保存成功')
    })
  })
}

onLoad()
</script>

```

- 该页面可集成到管理后台“系统设置”或“安全设置”模块。

- 支持LDAP配置的增/查/改，保存后后端自动生效。
- 可根据实际需求扩展字段和校验。

5. 典型问题与安全注意事项

- **LDAP注入防护**：所有用户输入必须转义，严禁拼接LDAP查询。
- **服务账号最小权限**：只读权限，限制搜索范围。
- **加密通信**：生产环境必须用LDAPS或StartTLS。
- **账号锁定与同步**：本地账号锁定/禁用时，LDAP用户也应受控。
- **敏感信息保护**：服务账号密码用环境变量/密钥服务管理。

6. 方案总结

本方案实现了Spring Boot应用对企业LDAP的无缝集成，兼容本地和LDAP账号，支持用户信息与权限自动同步，安全合规，便于后续扩展。开发时建议先在测试环境联调，确保LDAP配置和映射逻辑正确后再上线。

本方案不仅支持登录时的即时同步，还支持通过定时任务实现本地与LDAP目录的全量信息和注销同步，确保用户信息、账号状态与企业目录高度一致。

本方案支持通过数据库和管理后台灵活配置LDAP参数，适合单企业私有化部署，后续可平滑升级为多租户架构。